

PROACTIVE CLOUD AUTOSCALING

Reliability-Aware Hybrid Autoscaling

Integrating Forecasting and AIOps with LLM Guardrails

Sufiya Rahemtulla, Mustafa Daraghmeh, and Dariush Ebrahimi

Wilfrid Laurier University · Waterloo, Ontario, Canada

The Autoscaling Reaction Gap



Reactive autoscalers (e.g. Kubernetes HPA) act only AFTER utilization rises.

This leaves a reaction gap where SLA violations accumulate during sudden spikes.

Three gaps in prior proactive work



Misaligned Metrics

Optimizing average error (RMSE) masks catastrophic tail-event failures.



Latency Realism Gap

Models assume instant provisioning — ignoring container startup delays.



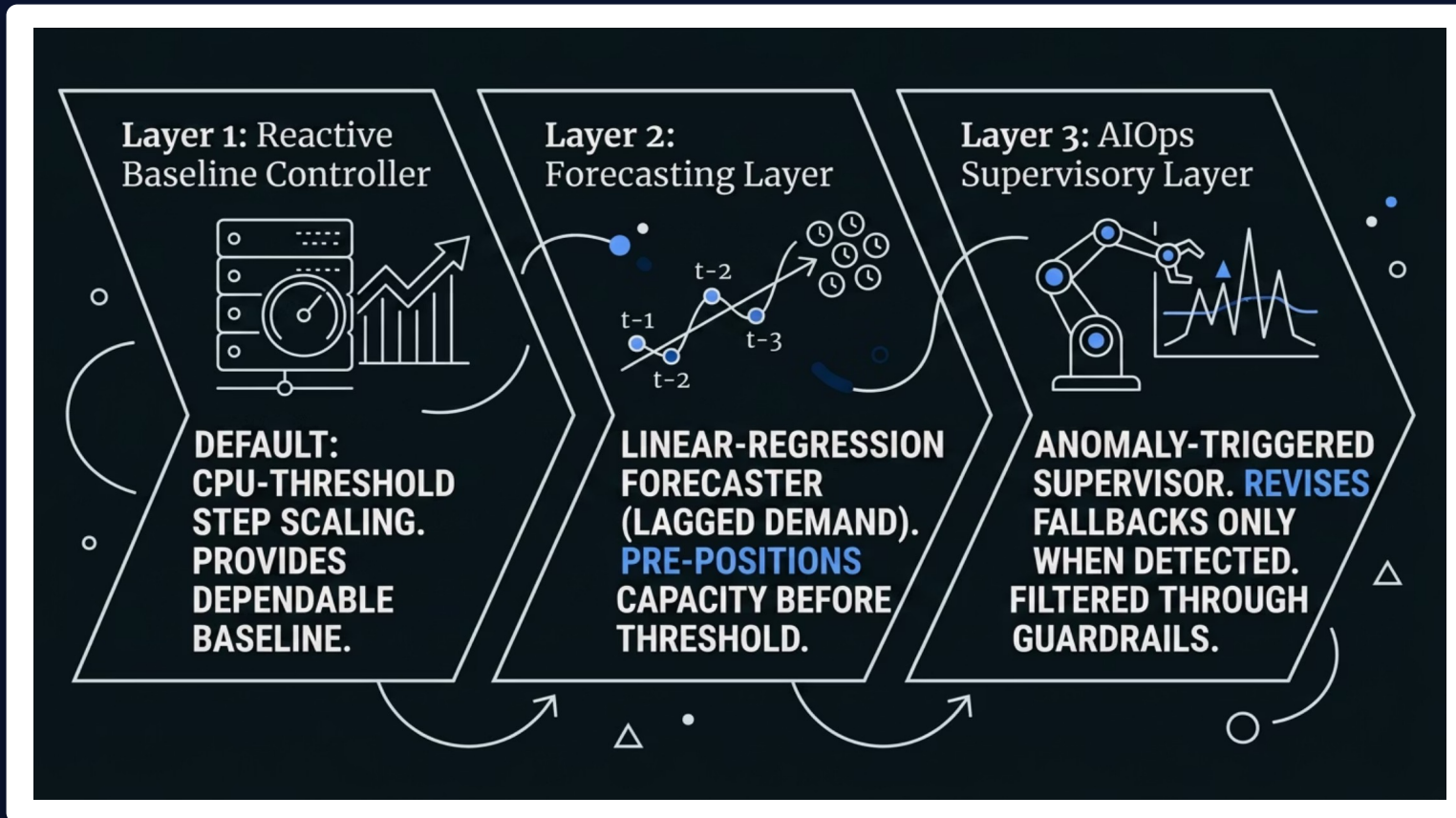
Ungoverned Autonomy

RL & LLMs are erratic and non-deterministic, unbound by cost or safety.



Our answer: a governed, three-layer hybrid framework that delivers responsiveness without sacrificing operational safety.

Framework: Three Interacting Layers



Layer 1 • Reactive Baseline

Dependable CPU-threshold step scaling — the safety net.



Layer 2 • Forecasting

Lightweight linear regression pre-positions capacity.



Layer 3 • AIOps Supervisor

Asynchronous, anomaly-triggered contextual advisor.



Deterministic Guardrails

AI never scales directly — every action is filtered.

Forecasting: Operationally Useful Prediction

Philosophy: judge forecasts operationally, not just statistically — maximize reaction-gap reduction at negligible inference cost.



ARIMA (5,1,0)

REJECTED

MAE 131.86 — flat trajectory, slow to burst onset.



Random Forest

REJECTED

No operational gain to justify added complexity.



Linear Regression

SELECTED

Lowest MAE 97.46 — preserves short-horizon direction.

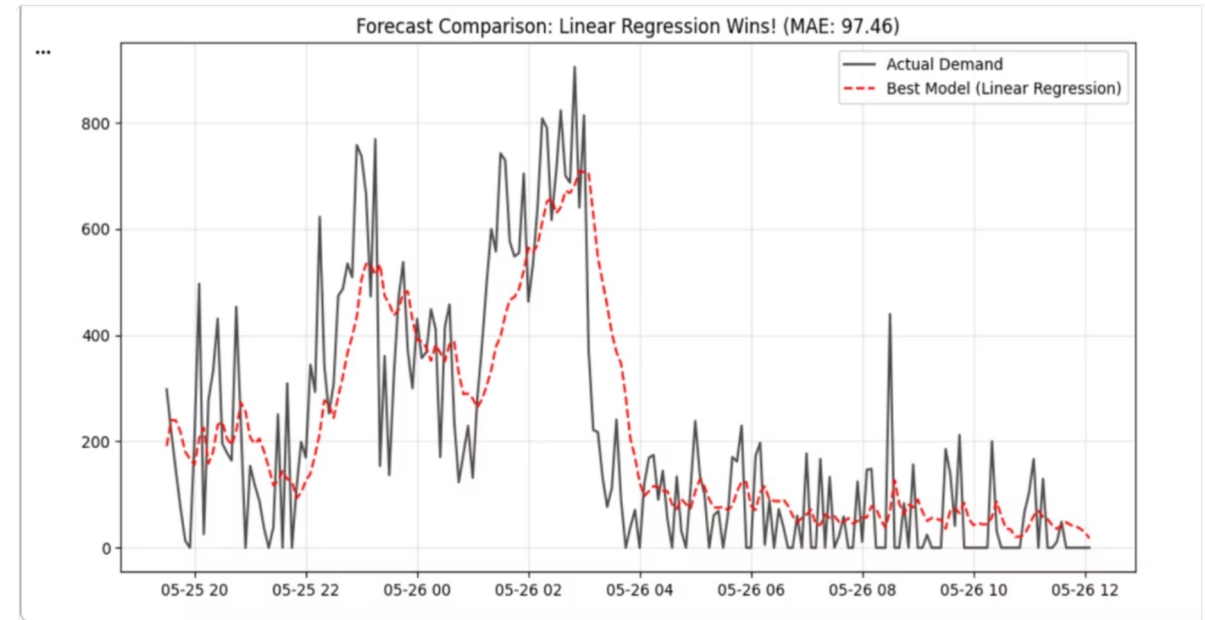


Fig. 1 Linear-regression forecast vs. observed demand — lag features track burst buildup.

AIOps Supervision & Guardrails



Anomaly-triggered only

Fires when forecasting residual exceeds 2σ of rolling error — focused on high-impact anomalies.



Structured recommendation

Runtime context is serialized to a compact JSON payload for Gemini 2.5 Flash, which returns a structured action.



Fail-safe default

If a recommendation fails ANY check, it is discarded and the system reverts to its last validated policy state — never a risky one.

Three sequential deterministic guardrails

01

Schema Validation

Malformed, missing-field, or API-error outputs are discarded.

02

Action Whitelisting

Only lower threshold, decrease cooldown, or take no action.

03

Value Bounding

Fallback thresholds clamped to [50, 95] to prevent over-scaling.

Experimental Methodology



Simulation environment

- Trace-driven simulator, OpenAI Gym-style interface
- Continuous 30-day horizon (8,640 timesteps)



Data & workloads

- Google Cluster-Usage Traces v3 (production)
- 3 profiles: Steady · Intermediate · Bursty (6.10 peak-to-mean)

Four progressive configurations

01

Reactive HPA

Standard threshold control
(baseline)

02

Predictive (ML only)

HPA + linear-regression forecast

03

Hybrid + Guardrails

Full proposed framework

04

Hybrid, No Guardrails

Ablation — validation removed

Configuration 04 is a structural ablation — it isolates the exact cost-reliability value of the guardrails.

FINDINGS

Results: Breaking the Trade-Off

85.2%

SLA violation reduction
14.2% → 2.1%

8×

Faster provisioning
120s → 15s

8.4%

Resource waste
Lowest viable config

Configuration	SLA Viol. (%)	Waste (%)	Latency (s)
Reactive Baseline (HPA)	14.2	12.0	120
Predictive (ML only)	8.5	15.5	45
Hybrid + Guardrails	2.1	8.4	15
Hybrid, No Guardrails	1.2	48.7	10

The trade-off, broken: the guarded hybrid achieves the lowest viable waste AND near-lowest violations. (Removing guardrails reaches 1.2% SLA — but waste explodes to 48.7%.)

Why Guardrails Are Structural

Removing guardrails looks tempting...

SLA violations drop to an all-time low of 1.2%.

...but resource waste explodes to 48.7%.



What happened

Under an extreme burst, the unconstrained LLM broke the safe boundary and tanked the fallback threshold to 40% — triggering aggressive over-provisioning.



The lesson

AI tools over-correct for local, short-term relief unless physically caged by guardrails enforcing system-wide cost limits.

Unguarded AIOps output · extreme burst

```
{
  "root_cause": "Predictive model
    failed to forecast burst...",
  "recommended_action":
    "LOWER_REACTIVE_THRESHOLD",
  "new_fallback_threshold": 40,
  "new_cooldown": 300
}
// 40 is below the safe minimum of 50
```

Why it matters:

the cost model is asymmetric — 1 unit of unmet demand = 50× the cost of node-time. Only guardrails align non-deterministic reasoning with that objective.

Three Broader Claims

1



Reliability as a First-Class Objective

Tail events dominate operational risk.
Modest safety margins beat brittle efficiency optimized for the average case.

2



Interpretable, Well-Scoped Prediction

Careful feature engineering makes a simple, transparent linear model cheap, responsive, and governable enough for control.

3



Bounded AI Assistance

The LLM's value is contextual adaptation during anomalies — only when its influence is narrow, inspectable, and constrained.

Successful AIOps depends less on maximizing model autonomy and more on defining a narrow, inspectable interface.

CONCLUSION & FUTURE DIRECTIONS



Key Takeaway

LLM-assisted AIOps is most useful embedded inside a deterministic safety framework. Bounded contextual guidance is more credible and more operationally valuable than unrestricted AI autonomy.

14.2% → 2.1%

SLA violations

8×

Faster provisioning

8.4%

Resource waste

Future Directions

- Validate across multiple cloud traces & heterogeneous services
- Make guardrail policy adaptive — conditioned on workload regime
- Compare with non-LLM anomaly-policy learners (stronger guarantees)
- Production evaluation with asynchronous integration & stale telemetry